

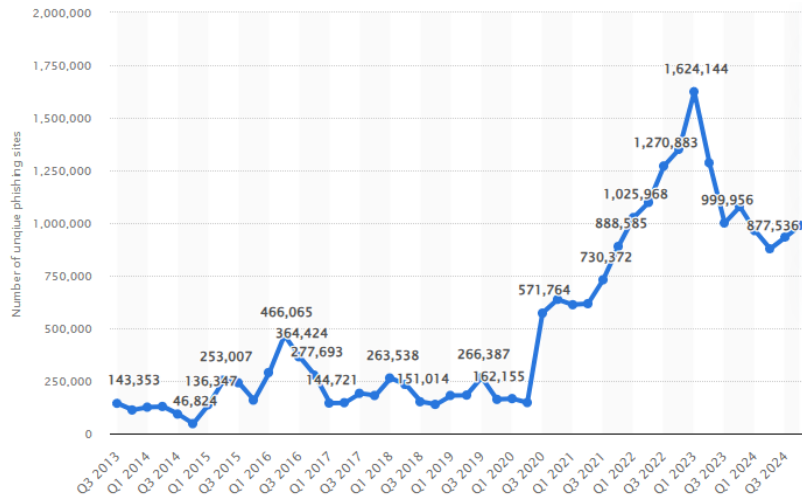
Rodrigo Arguello, Maurisa Dacosta, Nathan Poch
Professor Ben Ngan
CS 534: Introduction to Artificial Intelligence
06/11/2025

Project Proposal: Artificial Intelligence and Machine Learning Solutions to Phishing Detection

Rodrigo Arguello	Maurisa Dacosta	Nathan Poch
		
Masters in Computer Science	Masters in Computer Science	Masters in Cybersecurity

1. Introduction

Phishing, in cybersecurity, refers to an attempt by hackers to obtain a user's private information, such as passwords, credit cards, or bank account data (Burita et. al., 2021). Typically, an attacker will use email or sms to trick a user into clicking on harmful links or visiting harmful sites, and entering their credentials (GeeksforGeeks, 2024). Types of phishing include but are not limited to General Phishing, where an email is spoofed and sent to a large number of people, Spear Phishing, which is targeted at specific individuals working specific jobs at specific organisations (depending on the intended end goal), and Whaling, which is Spear Phishing targeting exclusively CEO's and other top executives (Rashid & Leyden, 2025). Phishing can target corporations or individuals, with consequences ranging from identity theft to data exfiltration, and often come at a financial cost to the victim (Admin, 2024). Email-based attacks lead to 91% of system breaches (Alkhalil et. al., 2021). Stolen credentials are the most common cause of data breaches, which cost victim organizations an average of over 4 million dollars (Imber, 2025). In 2015, a Whaling attack on FACC cost the aerospace company 47 million dollars (Imber, 2025). With an estimated 3.4 billion spam emails sent every day, and Google alone blocking approximately 100 million phishing emails daily, phishing is the most common form of cyber crime (Imber, 2025). With nearly one million unique phishing sites detected in the fourth quarter of 2024 alone, phishing remains a prevalent and dangerous form of social engineering (Statista, 2025).



The number of detected phishing sites across the world in each quarter from the 3rd quarter of 2013 to the 4th quarter of 2024 (Statista, 2025). The latest peak ended a sharp two year climb in Q1 2023, with 1.6 million unique phishing assaults. The 4th quarter of 2024 saw about five eighths as many, with only 0.989 million unique phishing sites (Statista, 2025). These numbers represent the number of unique base URLs of phishing sites (Statista, 2025).

Phishing attacks not only target world brands but also common civilians. The following examples of email scams after 2020 illustrate how quickly common people can lose their savings, security, confidence, and even their sense of online security. In Albuquerque, New Mexico-based 67-year-old restaurant server lost all of her \$32,000 retirement savings after falling for a phishing scam. In March 2023, she received a text message purported to be from Apple, informing her of an unauthorized purchase on her account. Panicked, she contacted the number given to her. A scammer (disguising himself as Apple support) made her believe that she had a compromised bank account and had to take all of her money and put it in a “safe” bitcoin account (kob4 news) . By the time it dawned on her what was actually happening, all of her money was gone. She reported to local police and to the FBI but was told by authorities no hope of recovery existed. This particular case highlights how fear and a sense of urgency can manipulate even cautious people to suffer severe financial loss. Also, an Australian couple, Sarah and Laine Robinson of the Gold Coast, were finalizing on a home purchase in late 2024 when they fell for an extremely targeted spear-phishing scam (yahoo! News). The Robinsons had emailed each other back and forward with their conveyancer on payment particulars. Without them realizing it, fraudsters had either hacked or spoofed their conveyancer’s email account. They received emails appearing genuine – down to accurate duty and fees – instructing them where to transfer their outstanding deposit. There was just one subtle red flag: absent “.au” at end of firm’s email address in reply address line. Thinking the email was on the level, the couple made two bank transfers of approximately A\$312,000 combined, only to discover on settlement eve that A\$252,000 had found its way to a scammer’s bank account. The spear-phishing email pretending to be their conveyancer had contained bogus account details and sent the money to the thief. Not only did their ideal home go but their life savings too, with some monies only recovered in part. Queensland Police followed the monies to a university student’s bank account in another state and presented how organized crime gangs recruit money mules. The example highlights today’s sophisticated phishing scams – well-educated individuals too fall for customized realistic-looking email in big-ticket transactions. It prompted Australian general advice to examine email addresses at least twice and to verify destinations of payment by contacting people they know via trusted mediums like phone or messaging.

The three SOTA methods that were chosen as the most effective to address our problem are: XGBoost, Random Forest, and MLP (Multilayer Perceptron). Despite similarities between XGBoost and Random Forest, we justify the exploration of both by their score similarity in previous studies (Singh & Maurya, 2024): we aim to replicate that result, and compare it to the MLP models. XGBoost (Extreme Gradient Boosting) is an optimized implementation of the gradient boosting algorithm, known for its high performance and efficiency (Shahrivari, Darabi, & Izadi, 2020).

It builds an ensemble of weak learners, typically decision trees, and incrementally improves them to form a strong predictive model. One of the key advantages of XGBoost is its scalability; it can handle large-scale datasets with billions of examples, making it suitable for both distributed systems and memory-constrained environments (Shahrivari, Darabi, & Izadi, 2020). Its support for parallel and distributed computing significantly accelerates the training process, enabling faster experimentation and model refinement. These reasons—along with its ability to utilize faster training and a regularization parameter that effectively reduces variance, helping to avoid overfitting—make it well-suited for phishing detection (Gu & Xu, 2022). Its use of a learning rate improves its generalization ability, which is beneficial when dealing with noisy or imbalanced datasets common in phishing scenarios, and its flexibility allows for more precise performance optimization.

Random forest is a supervised machine learning method that has been widely utilized in classification and regression problems (Dinesh et al., 2023). It combines multiple decision trees to improve prediction accuracy and reduce overfitting by allowing the trees to correct each other's errors. It also handles large datasets well and estimates generalization error effectively. Random forest will be effective for phishing detection because it is highly accurate, robust against noise and outliers, and performs well with imperfect data which is common in phishing scenarios. Its ability to reduce variance through ensemble averaging, as explained by the Central Limit Theorem, and its implicit feature selection make it both efficient and reliable for identifying complex patterns in phishing emails (Shahrivari, Darabi, & Izadi, 2020). The Multilayer Perceptron (MLP) model is a widely used light weight feedforward neural network in machine learning, serving purposes such as classification, non-linear pattern recognition and regression (Asani et al., 2024). MLPs are capable of extracting features from input data, identifying anomalies, and creating non-linear decision boundaries, making them especially suited for recognizing sophisticated phishing strategies that might evade simpler models (Reddy, 2023). Their adaptability allows them to continually learn from updated datasets, improving accuracy over time and reducing false positives, which contributes to a more reliable and responsive phishing detection system. The implementation of MLP supports real-time analysis which is crucial in phishing detection. With the proper parameter tuning and training the MLP model can be very successful.

To solve the problem of phishing emails, we will use the three SOTA methods discussed above to create a classification framework. This framework could then later be implemented into a real-time workflow environment, allowing for potential phishing emails to be flagged for IT review prior to being sent to employees.

To evaluate the effectiveness of these SOTA approaches, we will use the [Phishing Email Dataset](#) (Rokibulroni, 2025) (containing emails). Examples from the dataset, in tandem with our workflow, are discussed in section 3. The dataset will be split into training, testing, and validation sets, and then fed to DistilBERT, a more lightweight version of the self-attention-based BERT designed to be used in real-time scenarios, for feature extraction (Uddin et al., 2024). DistilBERT was chosen over BERT due to it being 40% smaller and 60% faster while still retaining 97% of the language understanding capabilities (Sanh et al., 2019). We will perform sample-by-sample mean pooling on DistilBERT's output to preserve computational efficiency and balanced representation (Xing et al., 2024). The resulting data will then be fed to the Random Forest, XGBoost, and MLP algorithms, which act as downstream classifiers. As in Owen & White's work from May of 2024, we will use Accuracy, False Positive Rate, and False Negative Rate, each defined below (Owen & White, 2024):

Key: TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$False\ Positive\ Rate\ (FPR) = \frac{FP}{FP + TN}$$

$$False\ Negative\ Rate\ (FNR) = \frac{FN}{FN + TP} = 1 - Recall$$

Owen & White also evaluate the model's Processing Latency, which is the time (measured in milliseconds) that it takes for the model to analyze incoming data and produce a classification result. We will simulate this by comparing the total testing time for each model on only the testing data.

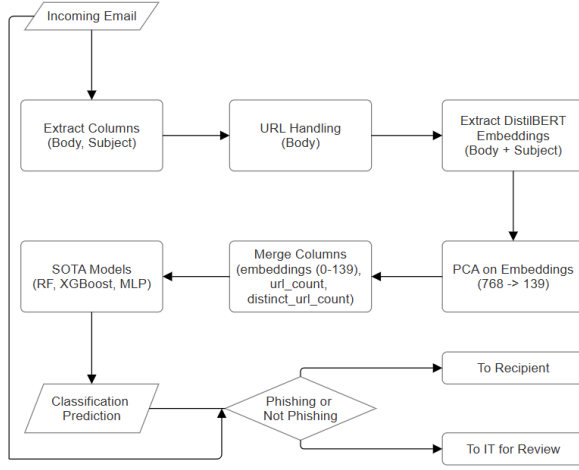


Figure 2

Figure 2 represents our methodology integrated into a real-world corporate setting. The incoming email is held while the data preprocessing and model predictions are carried out. If the model predicts that the email is not a phishing email, it is sent to the intended recipient. If the model instead predicts that the email is a phishing email, it is sent to the information technology (IT) department for review. IT then has the ability to analyze the email as necessary, and the authority to pass the email along to the recipient (if they deem the flag to be a false-positive), or drop the email (if they deem the flag to be true-positive).

For the purposes of testing our models, “Incoming Email” is replaced with our email dataset, and “Classification Prediction” is the predictions for each row in the dataset. “Phishing or Not Phishing”

becomes the calculation of our evaluation metrics, and, since we are testing the models, there is no pass through of emails.

2. SOTA Literature Review

Each of the selected models Random Forest, XGBoost, and MLP was particularly well-suited to the structure of our DistilBERT-transformed phishing dataset, which featured high dimensionality and inherent noise. These models are known for their capacity to manage complex feature spaces and generalize effectively in the presence of imperfect or imbalanced data.

Random Forest demonstrated strong performance through its ensemble approach, aggregating predictions from multiple decision trees trained on bootstrapped data samples. This method reduces variance, increases robustness, and naturally incorporates feature selection, making it well-aligned with the unpredictable and noisy nature of phishing data. Its ability to handle missing values and model non-linear decision boundaries without extensive preprocessing contributes to its reliability in dynamic, real-world environments.

XGBoost, with its iterative boosting framework, further enhanced predictive performance by correcting residual errors at each stage of training. Its built-in regularization mechanisms and support for parallelization offered both scalability and improved generalization—critical for detecting subtle variations between phishing and legitimate emails. The model’s flexibility in hyper parameter tuning and capacity to efficiently handle large-scale datasets made it especially advantageous for our use case, where rapid experimentation and refinement were essential.

The Multi-Layer Perceptron (MLP) is a feedforward neural network comprising an input layer, one or more hidden layers, and an output layer, whose neurons apply nonlinear activation functions and are trained using backpropagation, allowing them to classify complex, nonlinearly inseparable patterns. For phishing detection, MLPs have consistently outperformed traditional algorithms (e.g., SVM, KNN, decision trees) on primary metrics, such as accuracy, precision, recall, and F1-score, as demonstrated by PDMLP work (Al-Ahmadi et al., 2020) and others, which have yielded accuracies of ~96–98% and low false-positive rates. Compared to hand-engineered surface or feature-based models, MLPs automatically learn features, which helps them detect subtle anomalies within URL and text

information about phishing sites or emails. Their suitability for real-time deployment, gained through computational efficiency, and their ability to model nonlinear relationships enable them to combat phishing threats without relying on one-size-fits-all engineering methods. Their flexibility, good generalizability, and overall firm performance make MLP a state-of-the-art phishing detection method for phishing threats found today.

3. Proposed High-level Solution and Process

Figure 2 displays our methodology integrated into a real-world corporate setting. As discussed in section 1, the middle six boxes represent our workflow. In this section, we explain the function of those six boxes, with code snippets and examples.

This first box, “Extract Columns (Body, Subject)”, refers to the information we require from a given email in order to run our model. Four of our six datasets originally had 7 columns: *sender*, *receiver*, *subject*, *body*, *date*, *url*, *label*. The other two had 3 columns: *subject*, *body*, *label*. We chose to drop *sender* and *receiver* for the reasons below.

Reason	Explanation
Victim Pool (<i>receiver</i>)	Anybody and everybody can fall victim to phishing. One email receiving an abundant number of phishing attacks should not impact the model's calculations when analyzing an email from a known company. For example, suppose Microsoft sends out an email X to employees A and B, individually (meaning, both receive an identical email, but the emails are separate). Employee A has a history of receiving tons of phishing emails, while Employee B does not. If the model considers receiver email, and is trained on both, it is possible for it to allow the email through to Employee B, while blocking it from Employee A, which is not the intended functionality.
Email Spoofing (<i>sender</i>)	Phishing emails may not be sent from the same address recursively. Many sites exist to generate fake emails for short-term usage. While these sites are often used to sign up for services without disclosing the users true email, they can also be used to send emails without tracing it back to the true sender. In other words, while blocking an email may provide a temporary solution, it does not prevent the attacker from sending more emails in the future. Attackers can also take control of trusted emails and send spam from them. In other words, white-listing an email may provide attackers a method of bypassing the detection algorithm, making it better to identify spam from the content of the email, rather than the sender information.
Missing Data (<i>sender</i> , <i>receiver</i>)	Our final merged dataset contains approximately 80,000 columns. The 3-column sets made up for around 25,000 (31.25%) of these. Initial analysis of 10 randomly sampled rows from Ling.csv suggested that these datasets did not contain sender and receiver information in the body columns, leaving us with at least 25,000 Null

	values. This, combined with there already being missing values for these columns in the other 55,000 rows, could cause a substantial impact on the model training.
--	--

The *date* column was dropped because a) phishing attacks can be sent at any time, and b) we are not training a time-series model. The *url* column was a binary column indicating whether (1) or not (0) there was a URL in the *body* column. Since we wanted more information than this, and to handle URLs ourselves, we opted to drop this column and replace it with our own.

The second box, “URL Handling (Body)”, refers to the URL handling process the email body undergoes in our pipeline. We analyzed Nazario.csv’s first two rows to get an understanding of how best to handle URLs in our data. There are two primary methods of URL extraction: TLD locating and Regular Expression matching. Individually, these both have flaws. For example, the Regular Expression stored in *self.regex* in Code 1 will catch any URL beginning with *http://*, *https://*, *ftp://*, or *www.*, but will fail to recognize *google.com*. Jan Lipovský points this out in a post on their website, in which they also provide a potential alternative: URLExtract, a url extraction library in python that searches for URLs based on Top Level Domain (TLD) (“URLExtract - Introduction,” 2023). This method uses a public list of common top level domains to search a given text string for URLs, allowing it to catch *google.com* despite it not containing the beginnings mentioned above. However, it will fail to catch any *.cu.cc* domains, whether or not they lead with *http://* or something of the sort, since *.cu.cc* does not appear in the common domains list used for extraction. Both of these issues arose in Nazario row 1.

```
class url_extraction():
    ...

    A collection of functions for extracting URLs from text.
    ...

    def __init__(self):
        self.urlextract = URLExtract()
        self.regex = re.compile(r'((?:http|https|ftp):\/\/|www\.)(?:[a-z_-]+(?:[a-z_-]+)?)?(?:[a-z_.,@?^=%&\/~+#-]*[a-z_@?^=%&\/~+#-])')
        self.TRAILING_CHARS = '.,!~;:(){}<>|^\ \\'"
```

Code 1: A snippet from the beginning of our *url_extraction* class. Here, we see *self.urlextract* set up with Jan Lipovský’s URLExtract class, as well as *self.regex* being set up with a complicated regular expression for URL detection. By combining these two approaches, we can catch more URLs than either does alone.

To solve this problem, we opted to combine the two approaches, which required building a custom class that would independently use both. The results were stored in a dictionary of the form *{url: count}*. One concern with using both methods independently is that we may end up over-counting. For example, in the string *google.com*, *www.google.com*, *www.google.com*, both URLExtract and the regular expression method (henceforth RegEx) would come up with a count *google.com: 1*, *www.google.com: 2*, which, if not properly handled, would result in the dictionary *{google.com: 2, www.google.com: 4}*. To handle this, we first returned a list from RegEx, and then a separate list from URLExtract. Before combining the lists, the second list was compared to the first, and any entry of the URLExtract list that was already in the RegEx list was dropped. This can be seen in the *filtered_urlex_urls = ...* line of Code 2, which depicts our *string_extractor* function. Also seen in Code 2 is our *regex_cleaner*. Since RegEx can return unnecessary extra characters (for example, an extra */*), we need to clean the RegEx results prior to comparing them to the URLExtract results. Our two options for doing this were to use our own *self.TRAILING_CHARS* character set, or to use the cleaner that is already present in URLExtract. While both methods are built into the code, we opted to use the URLExtract cleaner for consistency.


```

def string_extractor(self, text, regex_cleaner='urlex'):
    """
    Extracts all URLs (that match certain criteria) from a string.
    We use both URLExtract and RegEx because URLExtract checks URL's based on TLD
    regex_cleaner takes the following values:
    - urlex
    - endchar
    They call their respective regex_<regex_cleaner>_extractor functions
    """
    url_list = []
    if regex_cleaner == 'urlex':
        url_list += self.regex_urlex_extractor(text)
    elif regex_cleaner == 'endchar':
        url_list += self.regex_endchar_extractor(text)
    else:
        raise ValueError('Invalid regex_cleaner. Please select either "urlex" or "endchar".')

    # Filter the urlex output to avoid false doubles
    # (regex and urlex both counting the same urls)
    urlex_urls = self.urlex_extractor(text)
    filtered_urlex_urls = [url for url in urlex_urls if url not in url_list]
    url_list += filtered_urlex_urls

    return url_list

```

Code 2: Our `string_extractor()` function, part of the `url_extraction` class. This function uses both URLExtract and RegEx to maximize the extracted URLs for our dataset.

Once extracted into a dictionary, the URLs in the body column were replaced with <url> tokens. We also generated two columns: `url_count` and `distinct_url_count`, which, respectively, contain the total number of URLs in a given email, and the total number of distinct URLs in the same emails. This allows our model to partially account for duplicity.

The third box, “Extract DistilBERT Embeddings (Body + Subject)”, is where DistilBERT is used. We combined the subject and body columns as `subject + “ ” + body`, and stored the results in a temporary text column. This text column was then passed to DistilBERT. We extracted 768 token features from each row using mean pooling to preserve computational efficiency and balanced representation (Xing et. al., 2024).

The fourth box, “PCA on Embeddings (768 → 139)”, is where we performed PCA on the DistilBERT pooled embeddings to reduce dimensionality. We used the `StandardScaler()` to normalize the data with a mean of 0 and a standard deviation of 1. We trained this scaler on our training data, and applied it to our testing and validation data. The same method was used with our PCA algorithm. We chose to keep enough dimensions to account for 95% of explained variance. This dropped the column count from 768 to 139, which is more handlable for our models. This dimensionality reduction was only done on the embeddings so that we could see, upon running our models, whether the URL columns we extracted were useful to our predictions.

The Fifth box simply represents the merging of the PCA Embeddings with our URL columns. This is here to clarify that PCA was run only on embeddings, and not on the URL columns. These columns are then passed to our SOTA models.

The sixth box introduces the application of our state-of-the-art(SOTA) machine learning methods to the processed feature matrix, which combines URL-based features with PCA reduced embeddings. This unified input serves as the foundation for training and prediction across all models. To effectively capture the diverse structural and semantic patterns in data, our team deployed three models: Multi-Layer Perceptron(MLP), Random Forest, and XGBoost. Each model utilizes this comprehensive feature set to learn complex relationships and improve phishing detection accuracy. Random Forest and XGBoost, with their ensemble based approaches and built in regularization, are well suited for handling noisy high-dimensional data, while MLP contributes non linear modeling capabilities that complement these strengths.

3.1 Random Forest

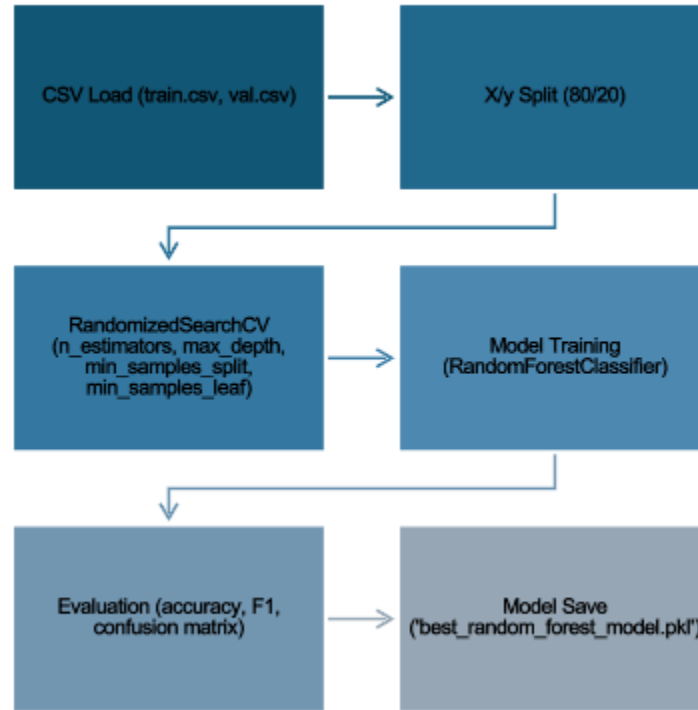


Figure 3 Pipeline Model

This pipeline was implemented using the following components:

- Scikit-learn: for model building (RandomForestClassifier), evaluation metrics, RandomizedSearchCV, and data splitting
- Scipy.stats: for sampling distributions during hyperparameter tuning
- Pandas, Numpy: for data loading and manipulation
- Matplotlib: for visualizing curves and confusion matrices
- Joblib: For saving the trained model to disk

Random Forest is implemented using RandomForestClassifier from the scikit – learn library. To optimize model performance, hyper parameter tuning is done with RandomSearchCV, which searches over key parameters such as:

- Number of trees (n_estimators)
- Maximum tree depth (max_depth)
- Minimum samples to split a node (min_samples_split)
- Minimum samples per leaf (min_samples_leaf)

These parameters are randomly sampled across predefined ranges and evaluated using 3 fold cross validation. The model is then trained with the best hyper parameters identified. This strategy takes into account an exploration of hyper parameters and the computational efficiency.

Pseudocode for Random Forest model with tuning:

```
#Random Forest Model with Parameter tuning using RandomizedSearchCV
rf_params = {
    'n_estimators': randint(300, 600),
    'max_depth': randint(5, 30),
    'min_samples_split': randint(5, 15),
    'min_samples_leaf': randint(2, 15),
}

rf_search = RandomizedSearchCV(RandomForestClassifier(random_state=42), rf_params, n_iter=10, cv=3, n_jobs=-1, verbose=1)
rf_search.fit(X_train, y_train)
```

Code 3: Hyper Parameters for Random Forest Model

Random Forest Model Equation Breakdown:

This explanation of the Random Forest algorithm, including bootstrap sampling and ensemble aggregation, is adapted from Ronaghan (2018).

Random Forest is an ensemble learning method that builds decision trees independently on bootstrapped samples of the training data. For each decision tree, Scikit-learn calculates a nodes importance using Gini Importance, assuming only two child nodes (binary tree):

$$ni_j = w_j C_j - w_{left(j)} C_{left(j)} - w_{right(j)} C_{right(j)}$$

- $ni_{sub(j)}$ = the importance of node j
- $w_{sub(j)}$ = weighted number of samples reaching node j
- $C_{sub(j)}$ = the impurity value of node j
- $left(j)$ = child node from left split on node j
- $right(j)$ = child node from right split on node j

The importance for each feature on a decision tree is then calculated as:

$$fi_i = \frac{\sum_{j: \text{node } j \text{ splits on feature } i} ni_j}{\sum_{k \in \text{all nodes}} ni_k}$$

- $fi_{sub(i)}$ = the importance of feature i
- $ni_{sub(j)}$ = the importance of node j

These can then be normalized to a value between 0 and 1 by dividing by the sum of all feature importance values:

$$normfi_i = \frac{fi_i}{\sum_{j \in \text{all features}} fi_j}$$

The final feature importance, at the Random Forest level, is it's average over all the trees. The sum of the feature's importance value on each trees is calculated and divided by the total number of trees:

$$RFfi_i = \frac{\sum_{j \in \text{all trees}} normfi_{ij}}{T}$$

- $RFfi_{sub(i)}$ = the importance of feature i calculated from all trees in the Random Forest model
- $normfi_{sub(ij)}$ = the normalized feature importance for i in tree j
- T = total number of trees

3.2 XGBoost Pipeline Model

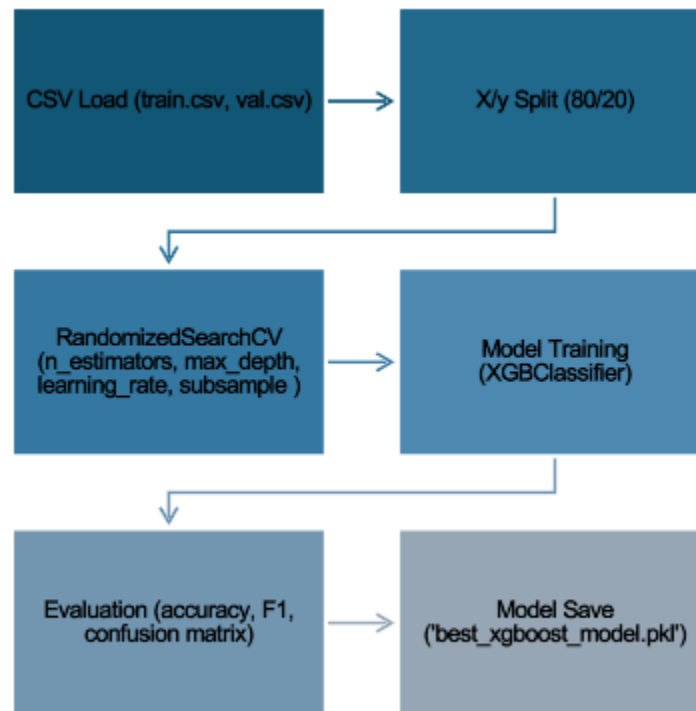


Figure 4 XG Boost Pipeline

This pipeline was implemented using the following components:

- Xgboost: for model creation and training (XGBClassifier)
- Scikit-learn: for evaluation metrics, RandomizedSearchCV, and data splitting
- Scipy.stats: for sampling distributions during hyperparameter tuning
- Pandas, Numpy: for data loading and manipulation
- Matplotlib: for visualizing curves and confusion matrices
- Joblib: For saving the trained model to disk

XGBoost uses XGBClassifier from the xgboost python library. To optimize model performance hyper parameter tuning was also done with RandomizedSearchCV as well, targeting

- Number of boosting rounds ($n_estimators$)
- Maximum tree depth (max_depth)
- Learning rate ($learning_rate$)
- Subsample ratio ($subsample$)

Parameters are randomly sampled across predefined ranges and evaluated with 3 fold cross validation. The model is then trained with the best found parameters, which encourages robust performance despite noisy or imbalanced phishing datasets.

Pseudocode for XGBoost model with tuning:

```
#XG Boost Model with Parameter tuning using RandomizedSearchCV
xgb_params = {
    'n_estimators': randint(100, 300),
    'max_depth': randint(3, 8),
    'learning_rate': uniform(0.01, 0.2),
    'subsample': uniform(0.6, 0.4),
}

xgb_search = RandomizedSearchCV(XGBClassifier(random_state=42, eval_metric='logloss'), xgb_params, n_iter=10, cv=3, n_jobs=-1, verbose=1)
xgb_search.fit(X_train, y_train)
```

Code 4: Hyper Parameters for XGBoost Model

XGBoost Model Equation Breakdown:

This explanation of the XGBoost algorithm, is adapted from GeeksforGeeks (2025).

The objective function in XGBoost consists of two parts: a loss function and a regularization term. The loss function measures how well the model fits the data and the regularization term simplify complex trees. The general form of the loss function is:

$$obj(\theta) = \sum_i^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Where:

- $l(y^{(i)}, \hat{y}^{(i)})$ measures the prediction error for the i^{th} instance
- $\Omega(f_k)$ is the regularization term which discourages overly complex trees.

Now instead of fitting the model all at once we optimize the model iteratively. We start with an initial prediction $\hat{y}^{(0)}=0$ and at each step we add a new tree to improve the model. The updated predictions after adding the t^{th} tree can be written as:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Where:

- $\hat{y}_i^{(t-1)}$ is the prediction from the previous iteration
- $f_t(x_i)$ is the prediction of the t^{th} tree for the i^{th} data point.

The regularization term $\Omega(f_t)$ simplify complex trees by penalizing the number of leaves in the tree and the size of the leaf. It is defined as:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Where:

- T is the number of leaves in the tree
- γ is a regularization parameter that controls the complexity of the tree
- λ is a parameter that penalizes the squared weight of the leaves w_j

Finally, when deciding how to split the nodes in the tree we compute the information gain for every possible split. The information gain for a split is calculated as:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

Where:

- G_L, G_R are the sums of gradients in the left and right child nodes
- H_L, H_R are the sums of Hessians in the left and right child nodes

Core functions Applied in both Random Forest and XGBoost:

```
#This curve gives a good idea of Overfitting or Underfitting between Training and Validation Data
def plot_learning_curve(estimator, X, y, title="Learning Curve", cv=5, train_color = "blue", 2 usages
                        val_color = "yellow", save_path = None):
    train_sizes, train_scores, val_scores = learning_curve(
        estimator, X, y,
        cv=cv,
        train_sizes=np.linspace( start= 0.1, stop: 1.0, num: 10),
        scoring='accuracy',
        n_jobs=-1,
        shuffle=True,
        random_state=42
    )

    train_mean = np.mean(train_scores, axis=1)
    val_mean = np.mean(val_scores, axis=1)

    plt.figure(figsize=(8, 6))
    plt.plot(*args: train_sizes, train_mean, label="Training accuracy", color= train_color, marker='o')
    plt.plot(*args: train_sizes, val_mean, label="Validation accuracy", color= val_color, marker='o')
    plt.title(title, color = "black")
    plt.xlabel("Training Set Size")
    plt.ylabel( ylabel: "Accuracy", color = "black")
    plt.legend(loc="best")
    plt.grid(True)
    if save_path:
        plt.savefig(save_path)
```

Code 5: Core Functions for XGBoost Model (1)

```

#Visually shows statistics of FPs, FNs, TPs, TNs
def confusion_bar(cm, title, colors, save_path= None): 2 usages
    tn, fp, fn, tp = cm.ravel()
    categories = ['True Negatives', 'False Positives', 'False Negatives', 'True Positives']
    values = [tn, fp, fn, tp]

    plt.figure(figsize=(8, 5))
    bars = plt.bar(categories, values, color=colors)
    plt.title(title, color = "black")
    plt.ylabel( ylabel: 'Count', color = "black")
    plt.ylim( *args: 0, max(values) * 1.1)

    for bar in bars:
        yval = bar.get_height()
        plt.text(bar.get_x() + bar.get_width() / 2, yval + max(values) * 0.02, int(yval), ha='center', va='bottom', color="black")

    if save_path:
        plt.savefig(save_path)

```

Code 5: Core Functions for XGBoost Model (2)

```

#Shows the performance of the parameter combinations
def plot_top_param_combinations(search, title, top_n=10, save_path= None): 2 usages
    results = pd.DataFrame(search.cv_results_).sort_values(by='mean_test_score', ascending=False).head(top_n)
    labels = [str(params) for params in results['params']]
    scores = results['mean_test_score']

    plt.figure(figsize=(12, max(6, top_n * 1.5)))
    bars = plt.barh(range(len(scores)), scores, color='blue')
    plt.yticks(range(len(scores)), labels, fontsize=8, color = "black")
    plt.xlabel( xlabel: 'Mean CV Accuracy', color = "black")
    plt.title(title, color = "black")
    plt.gca().invert_yaxis()

    for i, bar in enumerate(bars):
        width = bar.get_width()
        plt.text(width + 0.001, bar.get_y() + bar.get_height() / 2, s: f'{width:.3f}', va='center', color = "black")
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path)
    plt.show()

def save_CR(y_true, y_pred, filename): 2 usages
    report = classification_report(y_true, y_pred)
    with open(filename, "w") as f:
        f.write(report)

```

Code 5: Core Functions for XGBoost Model (3)

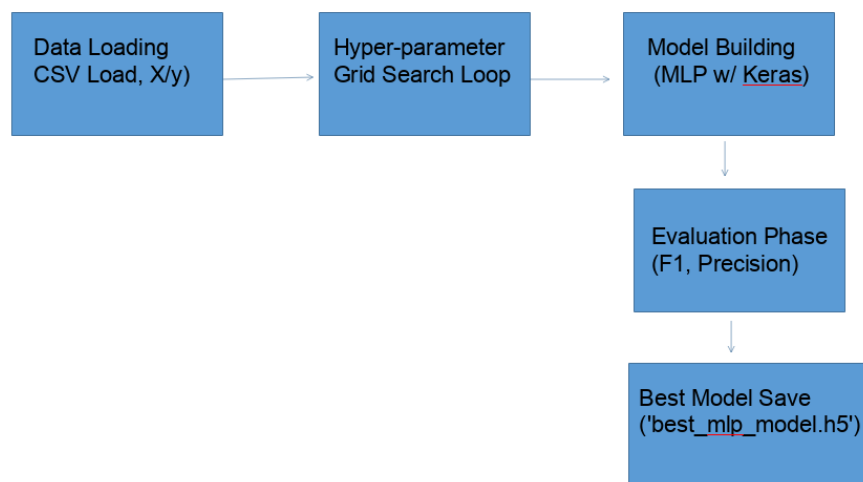
3.3 Multi-Layer-Perceptron Model

The phishing detection system utilizes a Multi-Layer Perceptron (MLP), a deep neural network that employs ReLU-activated hidden layers, which are well-suited to learning complex, nonlinear email patterns. For optimal training, overfitting is avoided by using EarlyStopping, which tracks validation loss, and training time is kept within five minutes by utilizing a special TimeStopping callback function. The performance of the model is evaluated using

the F1 Score, which is more effective at assessing imbalanced datasets since precision and recall are balanced. Grid search is finally used to sweep through optimal combinations of hyperparameters (such as layer structures, epoch numbers, and stopping conditions) to find the optimal model configuration.

The propose MLP Model uses the algorithm for grid search to optimize a Multi-Layer Perceptron (MLP) for phishing email detection. It systematically goes through all the hyperparameter combinations—layer settings, epochs, min loss improvement (min_delta), and patience. For each configuration, an MLP model is initialized and compiled, trained with early stopping as well as a time limit, and validated on both the validation and test datasets. F1 Score, Accuracy, and Recall are calculated. If a model achieves an F1 Score that is higher than the previously best-performing models, the previously best-performing models are updated. At the end of all runs, the best-performing model is saved and graphed for analysis.

MPL Model Pipeline



The pipeline was implemented using the following components:

Python Libraries:

- TensorFlow / Keras: For defining, training, and saving the MLP model.
- Scikit-learn: For evaluation metrics such as accuracy, precision, recall, F1, FPR, FNR.
- Pandas, Numpy: For data loading and manipulation.
- Matplotlib: For plotting learning curves.

Key Classes/Functions:

- Dense: For MLP layers with ReLU and sigmoid activations.
- EarlyStopping, TimeStopping: Training callbacks to stop based on criteria.
- eval_classification: Custom evaluation to compute all relevant metrics and visualize the confusion matrix.
- plot_history: Helper to visualize training/validation curves.
- Sequential: Model constructor for stacking dense layers.

Hyperparameter Search Space:

- Layer configs: [[64], [128, 64], [256, 128, 64]]
- Epochs: [50, 100]
- Min delta: [0.0, 0.001]
- Patience: [1, 3, 5]

Pseudocode for MLP Model

```
1. Initialize:
  - all_scores as empty list
  - best_f1 = -1
  - best_model = None
  - best_history = None
  - best_config = None

2. For each layer_config in layer_configs:
3.   For each epoch_count in epochs_list:
4.     For each min_delta in min_deltas:
5.       For each patience_value in patiences:

a. Build MLP model:
  - Add input and hidden layers based on layer_config
  - Use ReLU activation
  - Add output layer with sigmoid activation

b. Compile model with:
  - Adam optimizer (lr=0.001)
  - Binary crossentropy loss
  - Accuracy metric

c. Define EarlyStopping callback:
  - Monitor val_loss
  - Use min_delta and patience_value
  - Restore best weights

d. Define TimeStopping callback (5-minute limit)

e. Train model using:
  - training and validation data
  - specified epoch_count and batch size
  - early stopping and time limit

f. Log best validation loss

g. Plot training history (loss and accuracy)

h. Evaluate model on:
  - Training data
  - Validation data

i. Record metrics: Accuracy, Precision, Recall, F1, FPR, FNR

j. Append results to all_scores

k. If current val F1 score > best_f1:
  - Save model as best_model
  - Save history as best_history
  - Save config as best_config
  - Update best_f1

6. After loop:
  - Print best_config
  - Save best_model to disk
  - Save and sort all_scores
  - Plot history of best_model
```

MLP Model Equation Breakdown

F1-Scores

Precision: How many predicted phishing emails were actually phishing?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: How many actual phishing emails were correctly detected?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The core model is a binary classifier using Feedforward Neural Networks:

Let

$\mathbf{x} \in \mathbb{R}^n$

be the input feature vector.

Each hidden layer computes:

$$\mathbf{h}^{(i)} = \text{ReLU}(\mathbf{W}^{(i)}\mathbf{x} + \mathbf{b}^{(i)})$$

The final output is:

$$\hat{y} = \sigma(\mathbf{W}^{(o)}\mathbf{h} + \mathbf{b}^{(o)})$$

where σ is the **sigmoid** activation function for binary classification.

The **loss function** used is **binary cross-entropy**:

$$\mathcal{L}(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

MLP core Python functions

```
# Define EarlyStopping to stop training if val_loss doesn't improve
early_stopping = EarlyStopping(
    monitor='val_loss',          # Monitor validation loss
    min_delta=min_delta,        # Minimum change to qualify as improvement
    patience=patience,          # How many epochs to wait before stopping
    restore_best_weights=True,   # Restore weights from best epoch
    verbose=1,
    mode='min'                   # Looking for minimum val_loss
)

# Define TimeStopping to stop training if it exceeds time limit
time_stopping = TimeStopping(max_seconds=300)

# Print current model configuration
print(f"\nConfig: layers={layers}, epochs={epochs}, "
      f"min_delta={min_delta}, patience={patience}")

# Train the model with early stopping and time stopping callbacks
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=epochs,
    batch_size=32,
    callbacks=[early_stopping, time_stopping],
    verbose=2
)
```

```
# Track the best model based on validation F1 Score
val_f1 = val_scores.iloc[0]['F1 Score']
if val_f1 > best_f1:
    best_f1 = val_f1
    best_model = model
    best_history = history
    best_config = {
        'layers': layers,
        'epochs': epochs,
        'min_delta': min_delta,
        'patience': patience,
        'val_f1_score': val_f1
    }
```

```

# Custom callback to stop training after a time limit
class TimeStopping(Callback):
    def __init__(self, max_seconds=300):
        super().__init__()
        self.max_seconds = max_seconds
        self.start_time = None

    def on_train_begin(self, logs=None):
        self.start_time = time.time()

    def on_epoch_end(self, epoch, logs=None):
        if time.time() - self.start_time > self.max_seconds:
            print(f"\n⌚ Stopping training - time limit of {self.max_seconds} seconds reached.")
            self.model.stop_training = True

# Evaluation function
def eval_classification(model, X, y, name='model', labels=None):
    preds = np rint(model.predict(X))
    print(classification_report(y, preds, target_names=labels))

    ConfusionMatrixDisplay.from_predictions(y, preds, display_labels=labels)
    plt.show()

    tn, fp, fn, tp = confusion_matrix(y, preds).ravel()

    score_df = pd.DataFrame(index=[name])
    score_df.loc[name, 'Accuracy'] = accuracy_score(y, preds)
    score_df.loc[name, 'Precision'] = precision_score(y, preds)
    score_df.loc[name, 'Recall'] = recall_score(y, preds)
    score_df.loc[name, 'F1 Score'] = f1_score(y, preds)
    score_df.loc[name, 'FPR'] = fp / (fp + tn)
    score_df.loc[name, 'FNR'] = fn / (fn + tp)

    return score_df

# Function to plot training history
def plot_history(history):
    """Takes a keras model learning history and plots each metric"""
    metrics = history.history.keys()
    for metric in metrics:
        if not 'val' in metric:
            plt.plot(history.history[f'{metric}'], label=f'{metric}')
            if f'val_{metric}' in metrics:
                plt.plot(history.history[f'val_{metric}'], label=f'val_{metric}')
        plt.legend()
        plt.title(f'{metric}')
        plt.xlabel('Epochs')
        plt.ylabel(metric.capitalize())
        plt.grid(True)
        plt.show()

```

4. Preliminary Experimental Results and Discussions

To evaluate the effectiveness of our phishing detection pipeline, we conducted experiments using real-world email datasets containing labeled phishing and legitimate messages. These experiments assess the model's ability to classify emails based on features derived from PCA reduced DistilBERT embeddings and engineering URL indicators.

Random Forest and XGBoost

Performance was measured through a combination of classification metrics and diagnostic visualizations. Specifically, we evaluated model outputs using precision, recall, and F1-score for each class, along with macro and

weighted F1-scores to measure how well the model performs on both phishing and non-phishing emails, and how fairly it handles differences in their frequency. In addition we computed the number of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) for Random Forest and XGBoost. These values came from the confusion matrix and helped us see what kinds of mistakes the model made and how often they occurred. Overall accuracy scores were used as a quick summary of how many emails each model classified correctly.

To monitor overfitting and generalization, we tracked the gap between training and validation accuracy using learning curves, which visualize how model performance improves while increasing training data (Figures 7 and 8). During hyperparameter tuning we tested various settings for Random Forest and XG boost to find the best balance between model complexity and accuracy. Visualizing results from the randomized search helped us spot the top parameter combinations and their accuracy (Figures 3 and 4). This information will help future fine tuning with grid search, which focuses more closely on promising parameter values for better performance. Confusion matrix bar charts complemented this by showing the distributions of correct and incorrect predictions, giving insight into the models' reliability distinguishing phishing from legitimate emails (Figures 5 and 6).

Metric	Random Forest	XGBoost
Top Params	n_estimators: 442 max_depth: 19 min_samples_split: 10 min_samples_leaf: 4	n_estimators: 245 max_depth: 7 learning_rate: 0.145 subsample: 0.845
Train / Val Accuracy	99.83% / 96.88%	100.00% / 98.14%
Precision	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
Recall	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
F1-Score	Macro: 0.97 Weighted: 0.97	Macro: 0.98 Weighted: 0.98
Overall Accuracy	96.88%	98.14%

***Class 0 represents legitimate data, Class 1 represents phishing data**

The results show that both Random Forest and XGBoost models performed strongly in classifying phishing emails. XGBoost achieved the highest validation accuracy at 98.14%, with Random Forest close behind at 96.88%. Precision, recall, and F1-scores were consistently high across both models, showing they handled both phishing and non-phishing emails well without favoring one over the other.

While both models had very high training accuracy, the small gaps between training and validation accuracy around 2.95% for Random Forest and 1.86% for XGBoost suggests only mild overfitting. The learning curves helped confirm this by showing that validation accuracy improved steadily with more training data and eventually leveled off, meaning the models generalized well to new examples.

To better understand each model's behavior, the visualizations of the confusion matrix bar plot showed a well-balanced distribution of true positives, true negatives, false positives, and false negatives, confirming that both

models made consistent and reliable decisions across classes. The relatively low number of false positives and false negatives supports the idea that the models can be trusted in real-world settings.

To tune performance, RandomizedSearchCV was used to test different combinations of key parameters. The visualizations of the top parameter sets helped us understand which values worked best and how they affected accuracy. For instance, deeper trees and more estimators improved Random Forest, while XGBoost benefited from a moderate learning rate and higher subsample ratio. These plots gave us helpful direction on how to fine-tune the models further.

Next, GridSearchCv will be used to focus more closely on the top-performing parameter ranges. Unlike randomized search, grid search tests all combinations in a smaller space, allowing us to improve accuracy and consistency even more. This next round of tuning will help make the models more reliable, especially when handling tricky or borderline emails that resemble real ones.

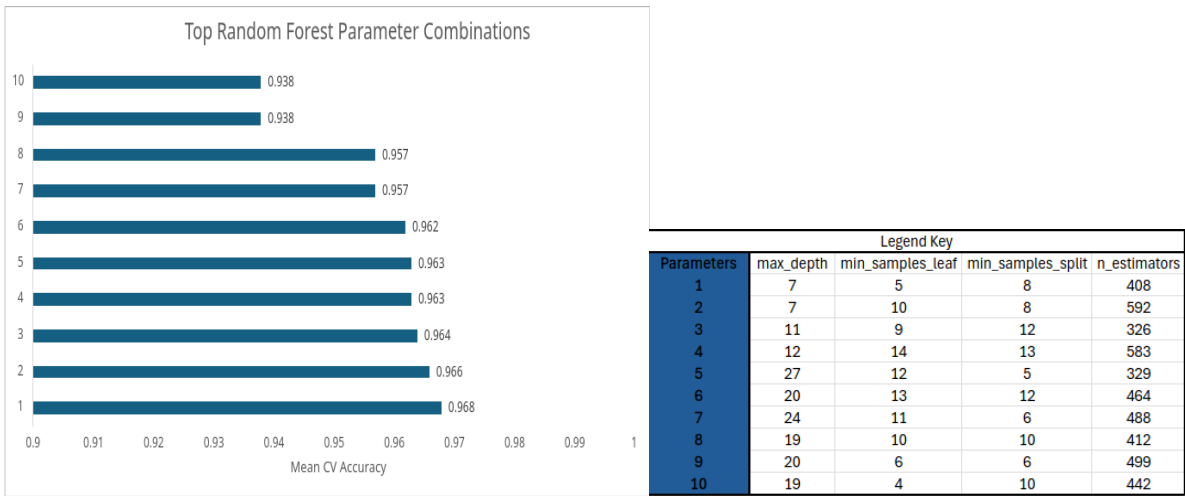


Figure 3:Top Random Forest Parameter Combinations – Hyperparameter sets ranked by mean cross-validation accuracy

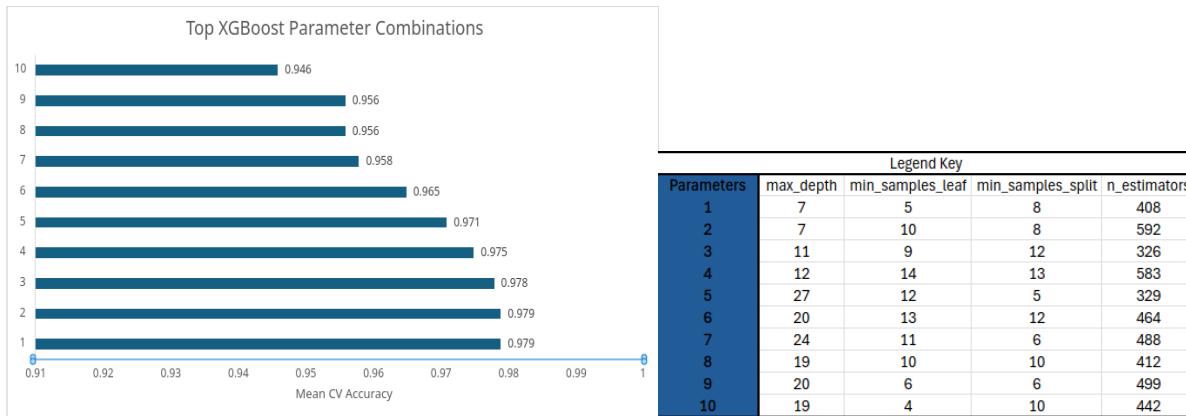


Figure 4:Top XGBoost Parameter Combinations – Hyperparameter sets ranked by mean cross-validation accuracy

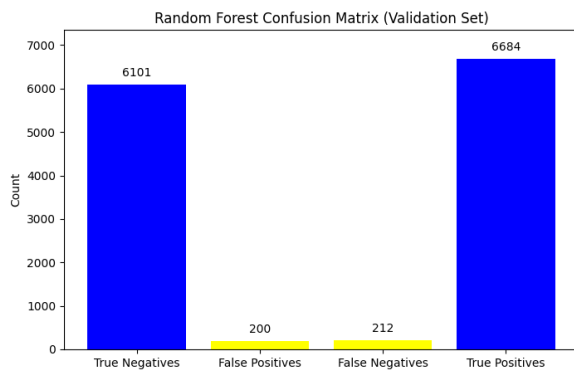


Figure 5: Random Forest Confusion Matrix: Bar chart breakdown of TP, TN,FP,FN on validation set

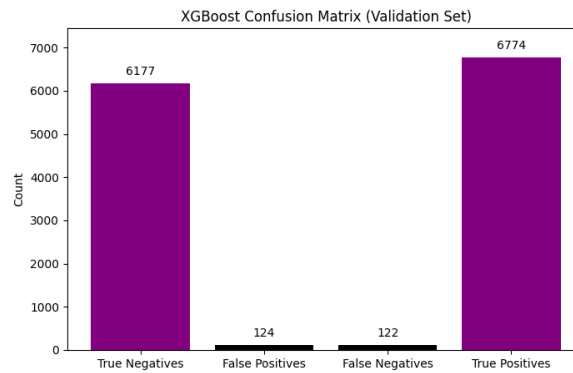


Figure 6: XGBoost Confusion Matrix: Bar chart breakdown of TP, TN,FP,FN on validation set

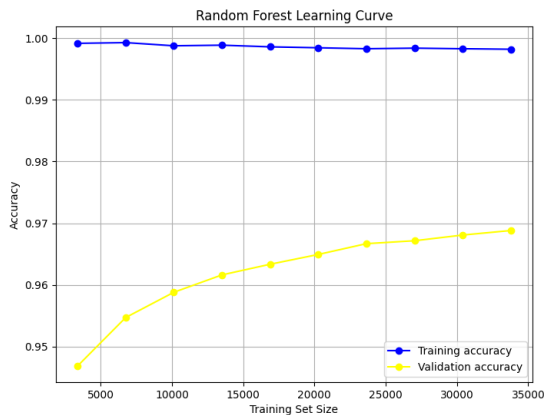


Figure 7: Random Forest Learning Curve: Training and validation accuracy trends across varying training sizes

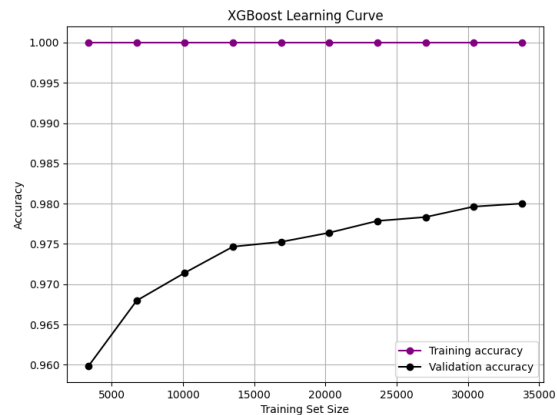


Figure 8: XGBoost: Training and validation accuracy trends across varying training sizes

MLP Model

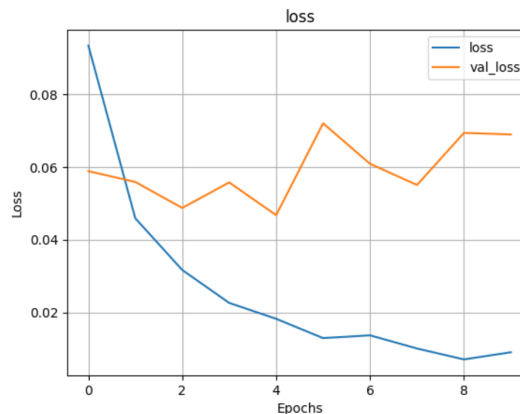
Our grid search yielded the best-performing MLP as a [128, 64] layer architecture, with 100 iterations, a minimum delta of 0.0, and a patience of 5. Its extremely high validation F1 score was 0.9877, establishing an excellent balance between precision (0.9876) and recall (0.9875). Saved under the name `best_mlp_model.h5`, it exhibits very low false positive (0.01317) and false negative (0.01247) rates. The top ten configurations—automatically sorted—also consistently achieved F1 scores above 0.985, demonstrating the robustness of the training process across hyperparameter variations. The loss curves shown exhibit evident convergence: the loss on the training set consistently dropped to zero, and the loss on the validation set converged after epoch ~5. The convergence indicates little overfitting, due to early stopping and time limits.

🏆 Best Model Configuration:
layers: [128, 64]
epochs: 100
min_delta: 0.0
patience: 5
val_f1_score: 0.9877438537965044

📁 Best model saved as 'best_mlp_model.h5'
Results saved to `mlp_model_results.csv`

📊 Top Validation Results by F1 Score:

	Accuracy	Precision	Recall	F1 Score	FPR	FNR
val [128, 64]-100-p5-d0.0	0.987194	0.987959	0.987529	0.987744	0.013173	0.012471
val [128, 64]-100-p3-d0.001	0.987043	0.987955	0.987239	0.987597	0.013173	0.012761
val [256, 128, 64]-100-p5-d0.001	0.986664	0.986252	0.988254	0.987252	0.015077	0.011746
val [256, 128, 64]-50-p3-d0.001	0.986436	0.987233	0.986804	0.987019	0.013966	0.013196
val [128, 64]-100-p5-d0.001	0.986285	0.985258	0.988544	0.986898	0.016188	0.011456
val [256, 128, 64]-50-p3-d0.0	0.985982	0.987081	0.986079	0.986580	0.014125	0.013921
val [256, 128, 64]-100-p3-d0.0	0.985830	0.984404	0.988544	0.986470	0.017140	0.011456
val [64]-100-p3-d0.001	0.985830	0.985948	0.986949	0.986448	0.015394	0.013051
val [256, 128, 64]-100-p5-d0.0	0.985527	0.986504	0.985789	0.986146	0.014760	0.014211
val [256, 128, 64]-50-p5-d0.0	0.985300	0.982853	0.989124	0.985979	0.018886	0.010876



To further regularize the model and guard against overfitting, we will consider adding:

- Dropout layers (e.g., 0.2–0.5) between hidden layers; dropout simulates model ensemble effects and improves robustness .

- L1 / L2 weight penalties in Dense layers to penalize excessive weights and encourage sparsity/smoothness.

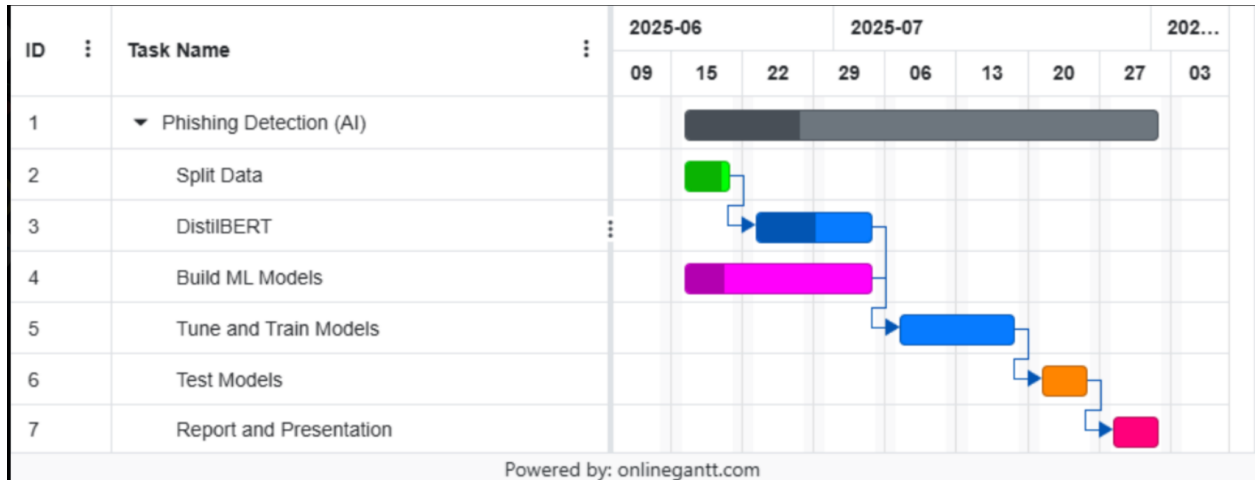
5. Lessons Learned

Maurisa : Coming from a chemistry background, this has been my first in depth experience with AI and machine learning. It has been exciting teaching myself how to build, train, and tune models as well as doing in depth research on what each hyperparameter means and how it influences a models performance. Also learning performance metrics and the significance behind them helped me broaden my technical skills. I developed a stronger understanding of a machine learning workflow and practiced executing these technical concepts in visualizations for presentations and documentation.

Nathan : My background in mathematics, cybersecurity, and machine learning has helped me focus in on some of the small details during our data cleaning phase. While I have learned about DistilBERT and how to use it to turn textual data into model-ready tokens, the biggest lessons to learn came from the URL extraction portion of our work. The Top Level Domain based approach offered by URLExtract provided an output on the example email I was analysing that differed from the regular-expression based method in CyberChef. This led me to build a class for extracting URLs that used both URLExtract and RegEx, giving me the best of both worlds, and serving as a reminder to always test the solutions provided by others. Also, while fixing a minor bug in one of the classes I built for this phase, I realized how convenient it was to fix bugs in a class one time as opposed to fixing the same bug in several locations in the code, providing further incentive to take the hard route the first time. Many of my math professors told me, “The great thing about mathematicians is that they are lazy.” This was in reference to making and proving theorems and propositions so that they would not need to solve the same problem over and over again. This bug fixing experience showed me the connection between theorem proving in math and class building in computer science, which I think was a valuable lesson in software engineering.

Rodrigo: Applying these AI and machine learning lessons can enhance manufacturing tests in aerospace by improving anomaly detection, automating data analysis, and optimizing test processes. Understanding model tuning, performance metrics, and modular code design enables smarter systems for identifying early faults, reducing test time, and increasing product reliability, ultimately ensuring higher safety and quality standards across complex aerospace production environments.

6. Timeline Schedule for the Remaining Project Tasks



Citations

Admin. (2024, April 23). *Phishing Attacks and Its Consequences - NTT Security - EN*. NTT Security - EN.

<https://se.security.ntt/en/understanding-the-consequences-of-a-phishing-attack/>

GeeksforGeeks. (2024, June 13). *What Is Phishing?* GeeksforGeeks. <https://www.geeksforgeeks.org/what-is-phishing/>

Imber, D. (2025, June 3). The Latest Phishing Statistics (Updated June 2025) | AAG IT Support. *AAG IT Services*. <https://aag-it.com/the-latest-phishing-statistics/>

Rashid, F., & Leyden, J. (2025, April 24). *9 Types of Phishing Attacks and How to Identify Them*. CSO Online. <https://www.csoonline.com/article/563353/8-types-of-phishing-attacks-and-how-to-identify-them.html>

Rokibulroni. (n.d.). *GitHub - Rokibulroni/Phishing-Email-Dataset: A Comprehensive Dataset of Phishing and Legitimate Emails Curated for Cybersecurity Research and Applications. This Dataset Is Designed to Help Researchers, Data Scientists, and Cybersecurity Professionals Develop, Train, and Evaluate Models for Phishing Detection, Email Filtering, and Threat Analysis*. GitHub.

<https://github.com/rokibulroni/Phishing-Email-Dataset/tree/main>

Statista. (2025, April 23). *Number of Global Phishing Attacks Q3 2013- Q4 2024*.

<https://www.statista.com/statistics/266155/number-of-phishing-attacks-worldwide/>

Tamal, M. (2023). Phishing Detection Dataset. *Mendeley Data*. <https://doi.org/10.17632/6tm2d6sz7p.1>

URLExtract - introduction. (2023, January 1). Jan Lipovský.

https://janlipovsky.cz/posts/urlextract_introduction/

Papers from Google Scholar and PapersWithCode

Amaz Uddin, M., & Sarker, I. H. (2024). An Explainable Transformer-based Model for Phishing Email Detection: A Large Language Model Approach. *arXiv e-prints*, arXiv-2402.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Singh, P., & Maurya, H. ML TECHNIQUES FOR PHISHING DETECTION.

Asani, E. O., Babalola, O. S., Akinola, A. E., Barnabas, A. S., Adams, O. V., & Odumesi, J. O. (2024, April). A server-side phishing detection api using long-short term memory (lstm) and multi-layer perceptron (mlp). In 2024 International Conference on Science, Engineering and Business for Driving Sustainable Development Goals (SEB4SDG) (pp. 1-8). IEEE.

REDDY, K. T. Enhancing Phishing Detection Through Multilayer Perceptron in Cybersecurity.

Shahrivari, V., Darabi, M. M., & Izadi, M. (2020). Phishing detection using machine learning techniques. *arXiv preprint arXiv:2009.11116*.

Dinesh, P. M., Mukesh, M., Navaneethan, B., Sabeenian, R. S., Paramasivam, M. E., & Manjunathan, A. (2023). Identification of phishing attacks using machine learning algorithm. In *E3S Web of Conferences* (Vol. 399, p. 04010). EDP Sciences.

Gu, J., & Xu, H. (2022, January). An ensemble method for phishing websites detection based on xgboost. In 2022 14th international conference on computer research and development (ICCRD) (pp. 214-219). IEEE.

Alkhalil, Z., Hewage, C., Nawaf, L., & Khan, I. (2021). Phishing attacks: A recent comprehensive study and a new anatomy. *Frontiers in Computer Science*, 3, 563060.

Burita, L., Matoulek, P., Halouzka, K., & Kozak, P. (2021). Analysis of phishing emails. *AIMS electronics and electrical engineering*, 5(1), 93-116.

Xing, J., Luo, D., Xue, C., & Xing, R. (2024). Comparative analysis of pooling mechanisms in llms: A sentiment analysis perspective. *arXiv preprint arXiv:2411.14654*.

Owen, A., & White, E. (2024). Evaluating the Effectiveness of AI-Based Phishing Detection in IoT Communication Protocols.

Al-Ahmadiet(May, 2020) Phishing Email Detection Model Using Deep Learning. (n.d.). *Encyclopedia of Computer Science*.

Ronaghan, S. (2018, May 11). *The Mathematics of Decision Trees, Random Forest and Feature Importance in Scikit-learn and Spark*. Medium. <https://medium.com/data-science/the-mathematics-of-decision-trees-random-forest-and-feature-importance-in-scikit-learn-and-spark-f2861df67e3>

GeeksforGeeks. (May, 2025). *XGBoost in Machine Learning*. <https://www.geeksforgeeks.org/machine-learning/xgboost/>